

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з практичної роботи 2

з дисципліни «Архітектура програмного забезпечення»

на тему «Архітектура відомих програмних систем»

Виконав:

ст. гр. ПЗП-23-3

Ромасенко С.О.

Прийняв:

ст. викл. Сокорчук І.П.

Харків 2026

1 ІСТОРІЯ ЗМІН

№	Дата	Версія звіту	Опис змін та виправлень
1	25.04.26	0.1	Створено розділ «Завдання»
2	27.04.26	0.2	Створено розділ «Опис виконаної роботи»
3	28.04.26	1.0	Додано результати реалізації та додатки, завершено основне оформлення
4	29.04.26	1.1	Додано таймлайн відео

2 ЗАВДАННЯ

Метою практичного заняття є ознайомитися з архітектурними підходами у сучасних складних програмних системах, проаналізувати їхню структурну організацію, принципи масштабування та взаємодію компонентів.

Відповідно до завдання необхідно:

1. Підготувати доповідь на тему: «Архітектура відомих програмних систем: Netflix»;
2. Обрати програмну систему для аналізу;
3. Розробити слайди презентації доповіді;
4. Записати відеозапис доповіді тривалістю 10-12 хвилин;
5. Опублікувати відео на YouTube з облікового запису @nure.ua;
6. Додати файл текстового звіту (UTF-8) та програмні приклади;
7. Експортувати звіт у PDF.

3 ОПИС ВИКОНАНОЇ РОБОТИ

3.1 Загальні відомості про систему

Назва системи: Netflix.

Розробник: Netflix, Inc.

Призначення: Глобальна стрімінгова платформа для доставки відеоконтенту у високій якості. Система обслуговує понад 260 мільйонів користувачів у 190 країнах, генеруючи значну частку світового інтернет-трафіку.

3.2 Архітектурний стиль системи

Обрана система Netflix базується на мікросервісній архітектурі (Cloud-Native). Обґрунтування: Перехід від моноліту до мікросервісів дозволив системі незалежно масштабувати окремі вузли, забезпечити високу відмовостійкість та прискорити цикл розробки нових функцій.

3.3 Основні компоненти системи

Система складається з понад 1000 мікросервісів, ключовими з яких є:

- Zuul (API Gateway): Динамічна маршрутизація та безпека.
- Eureka: Реєстрація та виявлення сервісів у хмарі.
- Cassandra/EVCache: Розподілене зберігання даних та кешування.
- Open Connect: Власна мережа доставки контенту (CDN).

3.4 Взаємодія компонентів

Взаємодія побудована на принципах REST та подієво-орієнтованої моделі. Запит клієнта проходить через Zuul, ідентифікується, після чого Eureka направляє його до відповідного мікросервісу. Для стабільності використовується механізм Circuit Breaker (Hystrix).

3.5 Масштабованість та надійність

Масштабованість досягається горизонтальним розширенням у хмарі AWS. Для перевірки надійності використовується Chaos Engineering (Chaos Monkey), що випадково вимикає сервіси для тестування здатності системи до самовідновлення.

3.6 Приклади програмної реалізації

Реалізацію фрагментів логіки взаємодії продемонстровано через використання патерну Circuit Breaker. Описано запити до ШІ для отримання прикладів коду, що демонструють принципи ізоляції збоїв у розподіленій системі.

4 ВИСНОВКИ

У результаті виконання практичного заняття було досліджено архітектуру програмної системи Netflix; визначено її архітектурний стиль та основні компоненти; проаналізовано механізми забезпечення масштабованості та відмовостійкості.

Також було закріплено навички роботи з GitHub-репозиторієм відповідно до встановлених вимог щодо іменування файлів та структури директорій (Pract2/).

Підготовлена відеодоповідь та презентація дозволили продемонструвати розуміння складних архітектурних рішень та принципів побудови високонавантажених систем.

ДОДАТОК А

Відеозапис доповіді на YouTube: https://youtu.be/example_link_pz2

Хронологічний опис відеозапису:

00:00 - Вступ, представлення автора та теми доповіді.

00:25 - Загальна характеристика системи Netflix та її масштаб.

01:07 - Аналіз архітектурного стилю: перехід до мікросервісів.

01:42 - Детальний огляд компонентів: Zuul, Eureka, Cassandra.

02:30 - Масштабованість: робота мережі Open Connect CDN.

04:00 - Приклади реалізації: Chaos Engineering та надійність.

4:36 - Висновки та результати аналізу.

ДОДАТОК Б

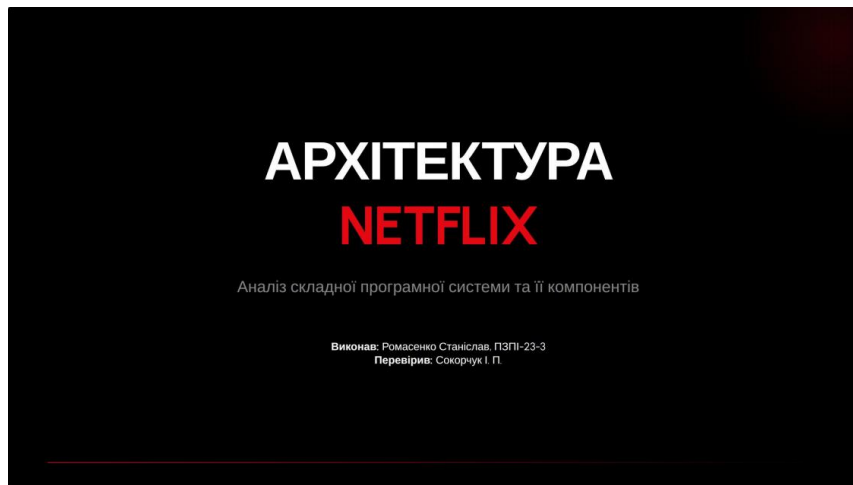


Рисунок Б.1 — Титульний слайд доповіді

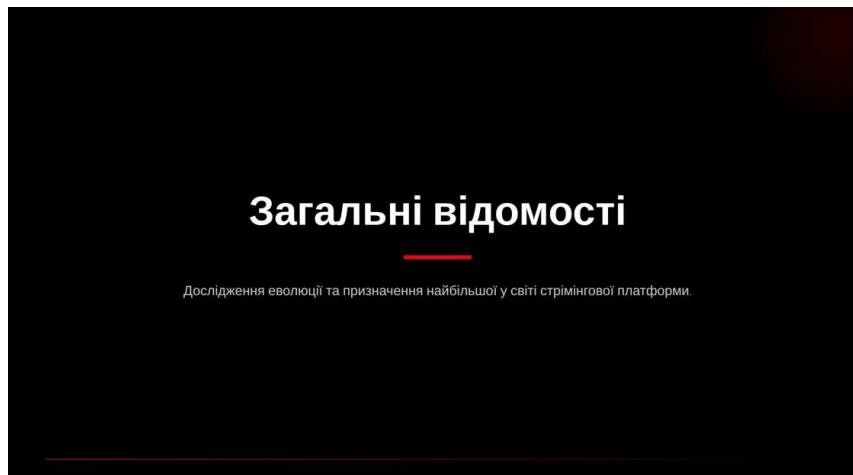


Рисунок Б.2 — Загальні відомості про систему

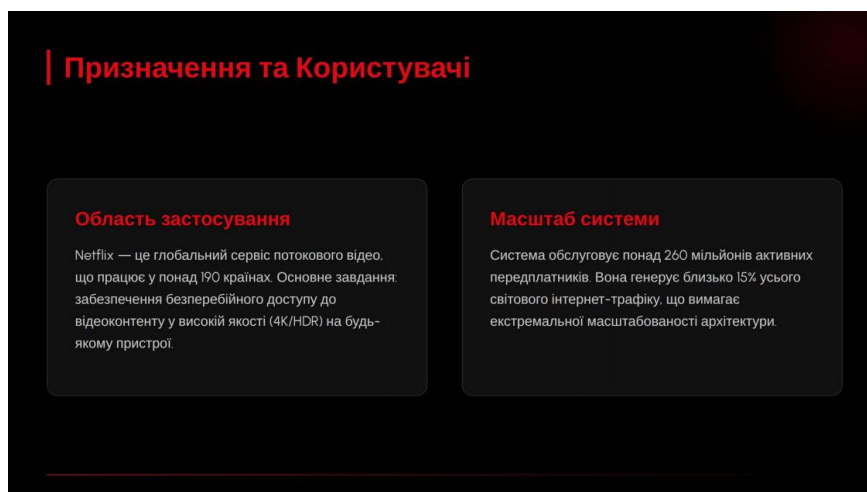


Рисунок Б.3 — Користувачі та призначення

| Архітектурний стиль: Мікросервіси

Netflix перейшов від моноліту до мікросервісної архітектури у 2008-2011 роках. Це дозволило:

- Незалежно розгортати понад 1000 сервісів.
- Масштабувати окремі вузли під навантаженням.
- Використовувати різні стеки технологій (Polyglot persistence).
- Забезпечити високу відмовостійкість всієї екосистеми.

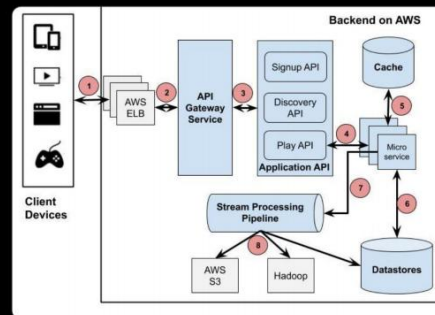


Рисунок Б.4 — Діаграма компонентів

| Ключові компоненти (Netflix OSS)



Zuul

API Gateway для динамічної маршрутизації, моніторингу та безпеки на вході до системи.



Eureka

Сервіс реєстрації та виявлення, що дозволяє мікросервісам знаходити один одного в хмарі.



Hystrix

Механізм переривання (Circuit Breaker) для запобігання каскадним збоєм у розподіленій системі.

Рисунок Б.5 — Ключові компоненти

| Open Connect CDN

Для мінімізації затримок Netflix побудував власну мережу доставки контенту (CDN) — Open Connect.

Спеціалізовані сервери (OCA) розміщуються безпосередньо у мережах інтернет-провайдерів (ISP) по всьому світу. Це дозволяє доставляти контент з максимально близької до користувача точки.

Рисунок Б.6 — Переваги архітектури

Порівняння архітектурних підходів		
Характеристика	Монолітна (до 2008)	Мікросервісна (Зараз)
Швидкість змін	Низька (місяці)	Висока (хвилини/години)
Масштабованість	Вертикальна (обмежена)	Горизонтальна (хмарна)
Відмовостійкість	Збій ламає всю систему	Ізольовані збої
База даних	Єдина централізована	Розподілена (Cassandra, EVCache)

Рисунок Б.7 — Недоліки та обмеження

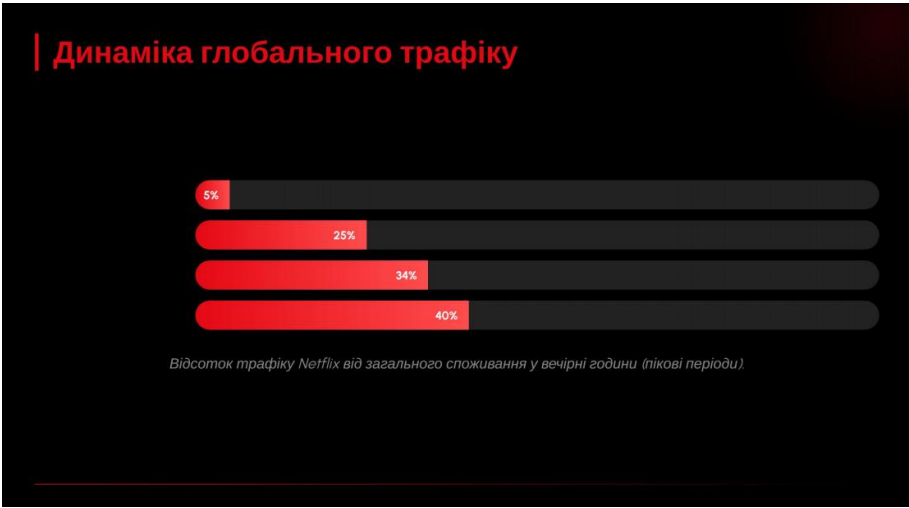


Рисунок Б.8 — Демонстрація коду

Надійність: Chaos Engineering

Chaos Monkey — інструмент, що випадково вимикає сервіси у продуктивному середовищі. Це змушує інженерів будувати архітектуру, здатну витримати будь-які збої.

$$R_{system} = \prod_{i=1}^n (1 - (1 - r_i)^{h_i})$$

Де R — надійність системи з надлишковими вузлами k.

Рисунок Б.9 — Висновок

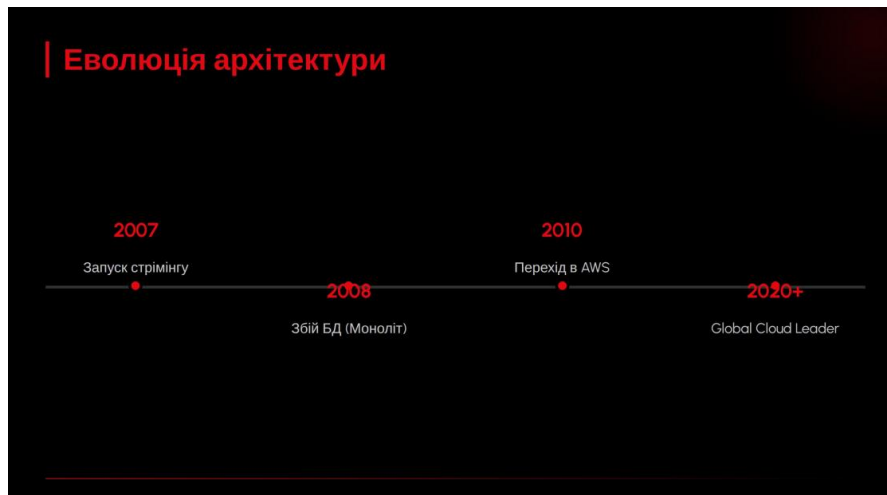


Рисунок Б.10 — Еволюція архітектури

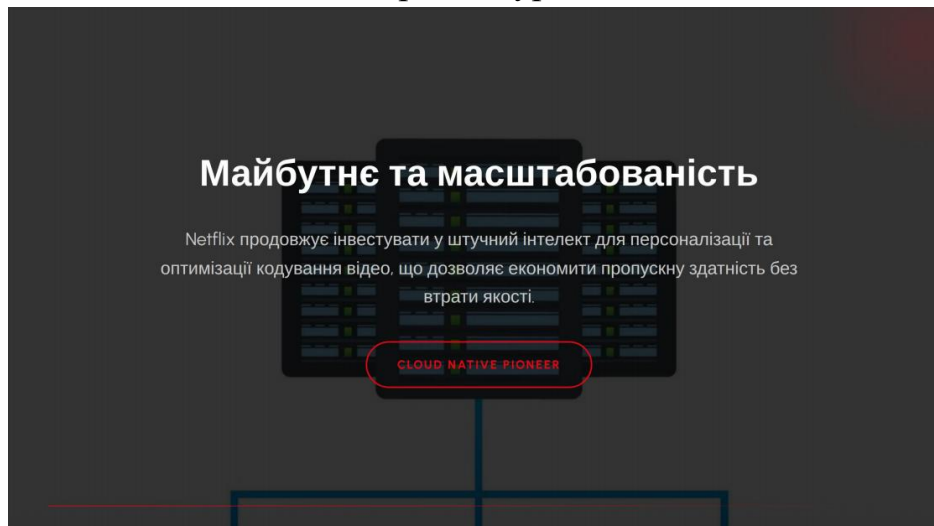


Рисунок Б.11 — Майбутнє та масштабованість

ДОДАТОК В

Запити до III: "Напиши приклад реалізації патерна Circuit Breaker на мові Java для виклику віддаленого сервісу в архітектурі Netflix."

"Оформи код із нумерацією рядків без підсвічування кольором для звіту за ДСТУ."

```
1 public interface VideoService {
2     String getStreamUrl(String videoId);
3 }
4 public class VideoServiceFallback implements VideoService {
5     @Override
6     public String getStreamUrl(String videoId) {
7         return "http://backup.cdn/static_video.mp4";
8     }
9 }
10 public class NetflixVideoProxy {
11     private VideoService remoteService;
12     public void setService(VideoService service) {
13         this.remoteService = service;
14     }
15     public String playVideo(String id) {
16         try {
17             return remoteService.getStreamUrl(id);
18         } catch (Exception e) {
19             return new VideoServiceFallback().getStreamUrl(id);
20         }
21     }
22 }
```

20 }

21 }

22 }